Université Grenoble Alpes
L3 Informatique

Année 2026-2027

Architectures logicielles et matérielles
Semaine 1 TD 2

L'objectif de ce TD est de réviser les opérations bit à bit et masquages, et les accès à des emplacements mémoire via leur adresse (pointeur).

I. Rappels.

Pour ***consulter*** *uniquement la valeur du* ***bit numéro k*** dans un entier naturel X, le principe est le suivant :
- créer un "masque" dont tous les bits sont à 0 sauf le bit numéro k
- "appliquer" ce masque à X, c'est à dire faire leur *et bit à bit*

```
      x_{n-1} x_{n-2} ...  x_k ... x_2  x_1  x_0
 &      0     0 ...      1 ...  0    0    0
```

=> si le résultat est nul, alors x_k est à 0, sinon x_k est à 1

Pour ***modifier*** *uniquement la valeur du* ***bit numéro k*** dans un entier naturel X (y placer une valeur v), le principe est le suivant :
- remettre à 0 le bit n°k (en utilisant un masque dont seul ce bit est à 0)
- puis placer la valeur voulue v (0 ou bien 1) dans X en utilisant un *ou bit à bit*

```
      x_{n-1} x_{n-2} ...  x_k ... x_2  x_1  x_0
 &      1     1 ...      0 ...  1    1    1
     ---------------------------------
      x_{n-1} x_{n-2} ...  0 ... x_2  x_1  x_0
 |      0     0 ...      v ...  0    0    0
     ---------------------------------
      x_{n-1} x_{n-2} ...  v ... x_2  x_1  x_0
```

Et pour ***consulter*** *uniquement la* ***valeur des bits numéro k1 à k2*** dans un entier naturel X (k1 < k2), le principe est le suivant :
- créer un "masque" dont tous les bits sont à 0 sauf les bits numéro k1 à k2
- "appliquer" ce masque à X, c'est à dire faire leur *et bit à bit*
- décaler le résultat à droite de k1 bits pour obtenir la valeur du nombre représenté par les bits numéro k1 à k2 de X

```
      x_{n-1} x_{n-2} ...  x_{k2} ... x_{k1} ... x_2  x_1  x_0
 &      0     0 ...      1    ...  1  ...  0    0    0
     ---------------------------------------------
      0     0  ...  x_{k2} ... x_{k1} ...  0    0    0

      0     0  ...  x_{k2} ... x_{k1} ...  0    0    0
 >>                                                k1
     ---------------------------------------------
      0     0  ...                  0  x_{k2} ...  x_{k1}
```

Exercice 1.

Q1. Ecrire **l'expression C** qui permet de fabriquer le masque sur 32 bits dont tous les bits sont à 0 sauf le bit d'indice k.

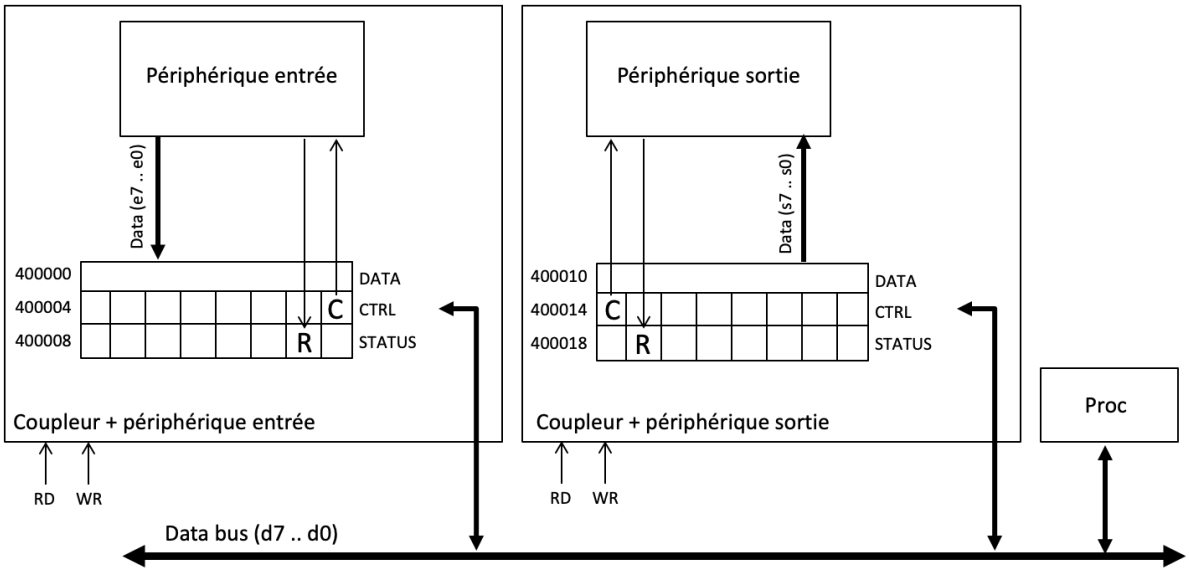
Q2. Ecrire **l'expression C** qui permet de déterminer si le bit d'indice k de la donnée contenue

dans une variable entier naturel sur 32 bits X est à 1.

Q3. (en devoir à la maison) Ecrire l'expression C qui permet d'obtenir (sur 32 bits) la valeur du nombre représenté par les bits d'indice k1 à k2 de la donnée contenue dans une variable entier naturel sur 32 bits X.

Exercice 2.

Nous considérons ci-dessous un système fictif dans lequel un coupleur d'entrée est relié à un périphérique d'entrée et un coupleur de sortie est relié à un périphérique de sortie :



Les données seront transférées (via le bus de données) octet par octet.
Le coupleur d'entrée peut récupérer un octet $e_7 \ldots e_0$ en provenance du périphérique d'entrée, pour le transmettre en mémoire.
Le coupleur de sortie peut transmettre des données en provenance de la mémoire vers le périphérique de sortie, sur $s_7 \ldots s_0$.

Les contenus des registres sont considérés comme des entiers non signés sur 8 bits.

Q1. Etant donné les adresses des différents registres données sur la figure, définir les **macros C** que nous pourrons utiliser dans la programmation des communications (penser à fournir la bonne information de typage) :

DATA_IN_ADR : adresse du registre DATA du coupleur d'entrée
CTRL_IN_ADR : adresse du registre CTRL du coupleur d'entrée
STATUS_IN_ADR : adresse du registre STATUS du coupleur d'entrée

DATA_OUT_ADR : adresse du registre DATA du coupleur de sortie
CTRL_OUT_ADR : adresse du registre CTRL du coupleur de sortie
STATUS_OUT_ADR : adresse du registre STATUS du coupleur de sortie

Q2. Nous supposons que, pour chaque périphérique, nous pourrons :
- savoir s'il est prêt pour une émission ou réception via un bit spécifique R (ready),
- et contrôler une opération spécifique via un bit C.

Dans **quel registre** se trouve le bit R ? (pourquoi ?)
Dans **quel registre** se trouve le bit C ? (pourquoi ?)
Repérer à quel **indice** du registre correspondant chacun de ces bits est stocké dans l'exemple traité ici (les bits de poids faible sont à droite).

Q3. Les valeurs de R ou de C devront transiter par le bus de données (8 bits) pour la communication avec le processeur. Toutefois chacune d'elles n'est codée que sur 1 seul bit !
L'octet transféré (contenu du registre correspondant) devra donc contenir **un 1 à l'emplacement associé à ce bit**, les autres bits n'étant pas significatifs ici.

Quelle **technique** devrons-nous utiliser pour ce faire ?

Définir en prévision **les macros** suivantes, définissant les **indices** respectifs :

```
#define CTRL_BIT_IN             // numero C du coupleur d'entree
#define STAT_BIT_IN             // numero R du coupleur d'entree

#define CTRL_BIT_OUT            // numero C du coupleur de sortie
#define STAT_BIT_OUT            // numero R du coupleur de sortie
```

Q4. En **utilisant les macros** définies aux questions 1 et 3, écrire les **expressions C** (on demande des expressions, pas des fonctions !) qui permettent de :

- **consulter** le bit R du périphérique d'entrée

- **mettre** dans le registre de contrôle du périphérique d'entrée **une valeur `v`** (0 ou 1) à l'emplacement de C (attention, on choisira ici de mettre les autres bits à 0)

- **consulter** le bit R du périphérique de sortie

- **mettre** dans le registre de contrôle du périphérique de sortie **une valeur `v`** (0 ou 1) à l'emplacement de C (attention, on choisira ici de mettre les autres bits à 0)

Q5. Si l'information de typage n'avait pas été utilisée dans les définitions de la question 1, comment faudrait-il alors par exemple écrire **l'expression** permettant de consulter le bit R du périphérique d'entrée ?

Q6. Ecrire les instructions qui permettraient de mettre dans le registre de contrôle du périphérique d'entrée **une valeur `v`** (0 ou 1) à l'emplacement de C, en laissant les autres bits de ce registre inchangés.